06-3-runtime-detection-and-runbook

accounts-api security assessment · generated from the repository, not hand-edited

> **Section 6 deliverable: a custom runtime detection rule written for this workload, with the alert payload, where it lands, who is woken, and the first three steps of the responder's runbook.**
>
> Assembled by `scripts/build-submission.py` from the repository. Source paths are given above each section.

`runtime/falco-rules.yaml`

```
# Custom Falco rules for accounts-api. Written for this workload, not lifted from
# the default ruleset — the default rules describe generic Linux badness, and on a
# cluster shared with a dozen teams that is where alert fatigue comes from.
#
# The design rule I applied: alert on things that are *impossible* in the
# remediated workload, not on things that are *suspicious* in general. A rule with
# a zero expected rate is a rule the on-call will still believe in six months.
#
# Threats: T2 (pod identity / lateral movement), T5 (data egress).

################################################################################
# scoping
################################################################################

- macro: accounts_api_container
  condition: >
    (container.id != host and
     k8s.ns.name = "accounts" and
     k8s.pod.label.app = "accounts-api")

# The single binary in the distroless image. PID 1 and nothing else.
- list: accounts_api_expected_binaries
  items: ["accounts-api"]

- macro: accounts_api_main_process
  condition: (proc.name in (accounts_api_expected_binaries) and proc.vpid = 1)

################################################################################
# RULE 1 — the one I would actually keep if I could only keep one.
#
# The database credential arrives as a file at a known path (Secrets Store CSI
# driver, k8s/externalsecret.yaml). In the remediated workload EXACTLY ONE
# process ever opens it: PID 1, the service binary, at startup and on rotation.
#
# Anything else reading that path means code is executing in the container that
# is not the service — an RCE payload, a debug shell, an injected library, or a
# sidecar somebody added without review. There is no benign version of this
# event, which is why it is CRITICAL and why it pages rather than tickets.
#
# This is deliberately narrower than the stock "Read sensitive file" rule: that
# one covers /etc/shadow and friends and fires constantly on package managers
# across the fleet. This one has an expected rate of zero.
################################################################################

- rule: accounts_api_credential_read_by_unexpected_process
  desc: >-
    A process other than the accounts-api service binary opened the mounted
```

```
    database credential inside an accounts-api container. In this workload the
    credential file has exactly one legitimate reader.
  condition: >
    open_read
    and accounts_api_container
    and fd.name startswith "/var/run/secrets/db/"
    and not accounts_api_main_process
  output: >-
    accounts-api database credential read by an unexpected process
    (threat=T2 rule_version=3
     file=%fd.name
     proc=%proc.name pid=%proc.pid vpid=%proc.vpid ppid=%proc.ppid
     cmdline=%proc.cmdline pcmdline=%proc.pcmdline exepath=%proc.exepath
     user=%user.name uid=%user.uid
     container=%container.id image=%container.image.repository:%container.image.tag
     digest=%container.image.digest
     pod=%k8s.pod.name ns=%k8s.ns.name node=%k8s.node.name
     start=%evt.time)
  priority: CRITICAL
  tags: [accounts-api, T2, credential-access, mitre_credential_access, T1552.001, page]

################################################################################
# RULE 2 — egress that the network layer let through.
#
# The NetworkPolicy and the Istio egress gateway (k8s/) permit exactly one
# external destination. This rule catches the cases the network layer cannot:
# traffic that leaves via the sidecar's allowed path but to a hostname the app
# resolved itself, and any direct socket that a NetworkPolicy misconfiguration
# would otherwise let through silently.
#
# Named in docs/05-egress.md as the compensating detection for what the egress
# control admits it cannot stop.
################################################################################

- list: accounts_api_permitted_egress_ports
  items: [443, 5432, 9094, 53]

- rule: accounts_api_unexpected_egress
  desc: >-
    accounts-api opened an outbound connection on a port outside its known set
    (443 to the Istio egress sidecar, 5432 to RDS, 9094 to MSK, 53 to CoreDNS).
    The workload speaks four protocols; anything else is an implant's channel.
  condition: >
    outbound
    and accounts_api_container
    and not fd.sport in (accounts_api_permitted_egress_ports)
    and not fd.dport in (accounts_api_permitted_egress_ports)
    and not fd.net = "127.0.0.0/8"
  output: >-
    accounts-api opened an unexpected outbound connection
    (threat=T5 rule_version=3
     dest=%fd.rip:%fd.rport proto=%fd.l4proto
     proc=%proc.name pid=%proc.pid cmdline=%proc.cmdline
     container=%container.id image=%container.image.repository
     digest=%container.image.digest
     pod=%k8s.pod.name ns=%k8s.ns.name node=%k8s.node.name
     start=%evt.time)
  priority: CRITICAL
  tags: [accounts-api, T5, exfiltration, mitre_exfiltration, T1041, page]

################################################################################
# RULE 3 — anything at all executing that is not the service binary.
#
# The image is distroless and readOnlyRootFilesystem is true, so there is no
```

```
# shell, no package manager and nowhere to drop one. Any exec is therefore either
# an attacker who brought their own, or an engineer who used `kubectl exec` on a
# production pod holding customer PII — both of which I want to know about, and
# the second of which is an access-review finding rather than an incident.
################################################################################

- rule: accounts_api_unexpected_process_spawned
  desc: >-
    A process other than the service binary executed inside an accounts-api
    container. The image is distroless with a read-only root filesystem; there is
    nothing legitimate to run.
  condition: >
    spawned_process
    and accounts_api_container
    and not proc.name in (accounts_api_expected_binaries)
  output: >-
    Unexpected process executed in accounts-api
    (threat=T2 rule_version=3
     proc=%proc.name exepath=%proc.exepath cmdline=%proc.cmdline
     parent=%proc.pname pcmdline=%proc.pcmdline
     user=%user.name uid=%user.uid
     tty=%proc.tty
     container=%container.id image=%container.image.repository
     pod=%k8s.pod.name ns=%k8s.ns.name node=%k8s.node.name
     start=%evt.time)
  priority: WARNING
  tags: [accounts-api, T2, execution, mitre_execution, T1059, ticket]
```

`runtime/runbook-accounts-api.md`

Runbook — `accounts_api_credential_read_by_unexpected_process`

A detection with no owner and no runbook is telemetry, not a control. This page is what makes the rule in `falco-rules.yaml` a control.

Alert payload

Falco emits JSON (`json_output: true`). The fields the responder needs, and why each is in the payload rather than something they have to go and look up at 03:00:

```
{
  "rule": "accounts_api_credential_read_by_unexpected_process",
  "priority": "Critical",
  "time": "2026-09-04T21:14:07.412Z",
  "tags": ["accounts-api", "T2", "credential-access", "T1552.001", "page"],
  "output_fields": {
    "fd.name":                 "/var/run/secrets/db/credential.json",
    "proc.name":               "sh",
    "proc.pid":                4471,
    "proc.vpid":               17,
    "proc.ppid":               1,
    "proc.cmdline":            "sh -c cat /var/run/secrets/db/credential.json",
    "proc.pcmdline":           "accounts-api --config /etc/accounts-api.yaml",
    "proc.exepath":            "/proc/self/fd/3",
    "user.name":               "app",
    "user.uid":                10001,
```

```
    "container.id":             "8f2c1ab77e04",
    "container.image.repository":"210987654321.dkr.ecr.ap-south-1.amazonaws.com/accounts-api",
    "container.image.digest":   "sha256:3f79b8e6…",
    "k8s.pod.name":             "accounts-api-7d9f4c8b5-x2klm",
    "k8s.ns.name":              "accounts",
    "k8s.node.name":            "ip-10-42-7-118.ap-south-1.compute.internal"
  }
}
```

Two fields carry most of the weight. **`proc.pcmdline`** says whether the service binary spawned it (RCE in the app) or whether the parent is a kubelet exec (a human with `kubectl exec`) — that single field decides which of the two branches in step 2 you are in. **`container.image.digest`** is what lets you pull the SBOM attestation and know what is in the image without guessing from a tag.

Where it lands and who is woken

```
falco (DaemonSet, ns/falco-system)
  └─ falcosidekick
       ├─ priority>=CRITICAL & tag "page"  → PagerDuty service "accounts-api-security"
       │                                       → security on-call (primary), 5-min escalation
       │                                         to platform-security lead
       ├─ all events                       → Slack #sec-alerts (thread per rule)
       └─ all events                       → CloudWatch Logs → OpenSearch (90d)
                                             → S3 log-archive account (7y, Object Lock)
```

Owner: Security Engineering on-call rota (PagerDuty schedule `security-oncall-ap-south-1`), **not** the accounts-api team. If the app is compromised, the app team is a stakeholder, not the responder — and in the branch where the alert is caused by an engineer running `kubectl exec`, the app team is the subject. Secondary: platform-security lead after 5 minutes. Business hours India, follow-the-sun after 20:00 IST to the London rota.

First three steps

1. Contain the credential, not the pod. (target: 5 minutes)

Do *not* kill the pod first. Killing it destroys the process tree and the attacker's foothold evidence, and the credential is already read — deleting the pod protects nothing. Instead trigger emergency rotation, which is the action that actually reduces harm:

```
aws secretsmanager rotate-secret \
  --secret-id prod/accounts-api/db \
  --client-request-token "inc-$(date +%s)"          # see docs/06-secrets-rotation.md
# then confirm the old version is revoked, not merely superseded:
aws secretsmanager list-secret-version-ids --secret-id prod/accounts-api/db
```

Rotation completes in ~90 seconds and the running pods pick up the new credential from the CSI mount without a restart — which is precisely why the manifest was changed away from `envFrom` in section 4a.

2. Decide which of two things this is, from the payload alone. (target: 10 minutes)

- `proc.pcmdline` is the service binary (`accounts-api …`) → **the application executed attacker-controlled code.** This is an incident. Declare Sev-1, page the accounts-api on-call as a stakeholder, and go to step 3.
- `proc.pcmdline` shows a kubelet/containerd exec parent, or `proc.tty` is set → **a human ran `kubectl exec` on a production pod holding customer PII.** Not (usually) an incident; it *is* an access-review finding. Correlate against the EKS audit log to name them: `bash aws logs filter-log-events \ --log-group-name /aws/eks/banking-shared-ap-south-1/cluster \ --start-time $(( ($(date +%s) - 900) * 1000 )) \ --filter-pattern '{ $.verb = "create" && $.objectRef.subresource = "exec" && $.objectRef.namespace = "accounts" }'` Then close the loop with their manager and, more usefully, ask why they needed a shell — that answer is usually a missing dashboard.

3. Scope the blast radius before eradicating. (target: 30 minutes)

Isolate the pod at the network layer rather than deleting it (keeps the process tree and memory available for forensics):

```
kubectl -n accounts label pod accounts-api-7d9f4c8b5-x2klm quarantine=true
# k8s/networkpolicy-quarantine.yaml selects quarantine=true and denies all ingress+egress
```

Then answer, in this order:

1. **What did the credential do?** RDS `pgaudit` /CloudWatch logs for that DB user since the alert timestamp — specifically any `SELECT` without a `WHERE` clause on `accounts`, which is the bulk-read signature.
2. **What did the pod identity do?** CloudTrail, filtered on the pod role's session — anything beyond `GetSecretValue` and `kafka-cluster:*` should be impossible per `iam/pod-identity-policy.json`, so a hit here is also a finding about that policy.
3. **Where did it come from?** `container.image.digest` → `cosign download attestation` for the SBOM → compare against the digest that `main` last built. A digest that is not in the ECR history means the admission policy was bypassed and this is a much bigger incident than one pod.

Escalate to the customer-data-breach process (72-hour DPDP notification clock, RBI incident report within 6 hours of classification) the moment step 3.1 shows a bulk read. That clock starts at *detection*, not at conclusion of the investigation — so start it in parallel, not afterwards.

Known false-positive sources

- A debug sidecar injected by the OTel operator (should be impossible in `ns/accounts` — the admission policy blocks unknown registries — but the operator has been known to patch pods post-admission).
- `kubectl debug` ephemeral containers during an availability incident. Legitimate and still worth the alert: the responder should know a shell exists in a pod with the credential mounted.

Expected rate: **zero**. If this rule fires more than twice a quarter without a declared incident, the rule is wrong or the workload changed — fix one of those rather than tuning the alert down.